

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Механико-математический факультет
Математика и компьютерные науки
Кафедра Программирования

РЕФЕРАТ
RENDEZVOUS CHANNELS

Студент: Жакулин Владимир Владимирович
Группа: 23126

Научный руководитель: Трепаков Иван Сергеевич
Индустриальный доцент, старший преподаватель
кафедры программирования ММФ

Реферат принят: _____ Трепаков И.С.
« _____ » _____ 2026 г.

Новосибирск
2026

Содержание

Содержание	1
1 Введение	2
2 Конфигурация	3
3 Алгоритм	4
3.1 Отправление сообщений	5
3.2 Получение сообщений	7
4 Измерения	9
4.1 Окружение	9
4.2 Бенчмарки	9
4.3 Результаты	9
4.4 Отравленные ячейки	9
4.5 Использование памяти	10
5 Заключение	11
Список литературы	12

1 Введение

Асинхронное программирование - это классическая конструкция, позволяющая разделить выполнение единицы кода от основного приложения. Оно полезно, к примеру, при выполнении кода с высоким разбросом задержки, такие как операции ввода-вывода (I/O) или сетевые запросы. Корутины представляют собой общий и эффективный способ поддержки асинхронного программирования. Они могут приостанавливать(suspend) и возобновлять(resume) выполнение кода, тем самым обеспечивая кооперативную(невытесняющую) многозадачность. Программирование с использованием сопрограмм похоже на стандартное многопоточное программирование, с важным отличием, что сопрограммы являются легковесными, а также переиспользуют нативные потоки. Это позволяет создавать сопрограмму для небольших задач, объединять их с помощью структурированной многопоточности. Другим отличием от стандартных потоков является, также, то, что сопрограммы не требуют дополнительной синхронизации, так как явно передают исполнение, вместо этого полагаясь на отправку сообщений.

Основным примитивом синхронизации для сопрограмм служит рандеву-канал. Рандеву-канал представляет из себя блокирующую очередь с нулевой ёмкостью. Он поддерживает запросы на отправку(send(message)) и на получение(receive()) сообщений. Отправление сообщений проверяет наличие в очереди получателя, либо удаляет одного из них, либо добавляет себя, как ожидающего отправителя. Получение сообщений работает симметрично. Операции являются, умышленно, блокирующими. Рандеву-канал является основным примитивом синхронизации в асинхронном программировании, выступая в качестве основы для более сложных протоколов передачи сообщений.

Таким образом, неэффективная реализация рандеву-каналов может привести к деградации производительности. В статье “Fast and Scalable Channels in Kotlin Coroutines”(Nikita Koval, Dan Alistarh, Roman Elizarov)[1] предлагается эффективный алгоритм реализации рандеву-каналов с измерениями.

2 Конфигурация

Для управлениями сопрограммами будет использоваться следующий интерфейс:

```
1 interface Coroutine {  
2     fun tryUnpark(): Boolean  
3     fun interrupt(): Unit  
4     fun park(onInterrupt: lambda () -> Unit): Unit  
5 }  
6  
7 fun curCor(): Coroutine
```

Листинг 1: интерфейс управления сопрограммами

Функция `curCor` используется для получения текущей исполняющейся сопрограммы. Метод `park` используется для приостановления сопрограммы. Сопрограммы также могут быть отменены с помощью метода `interrupt`. В случае отмены приостановленной сопрограммы выполняется обработчик `onInterrupt`, переданный во время остановки. Для того, чтобы возобновить сопрограмму предоставляется метод `tryUnpark`, возвращающий `true` при успехе, `false` если сопрограмма была отменена.

В системе предполагается наличие атомарных Compare-And-Swap(CAS) и Fetch-And-Add(FAA) инструкций. Алгоритм предлагается реализовать в среде выполнения с поддержкой сборки мусора.

3 Алгоритм

Предлагается реализовать рандеву-канал на основе структуры данных, называемой бесконечным массивом. В нём мы будем хранить сообщения и сопрограммы, со счётчиками выполненных отправок и получений. И отправление и получение в начале прибавляют свои счётчики, тем самым резервируя себе место в массиве. Каждая операция либо блокирует исполнение, либо проводит “рандеву” с запросом противоположного типа. Предполагается, что каждый элемент массива будет обработан ровно одной отправкой и ровно одним получением.

```
1 class Channel<Message> {
2     var S, R: Long
3     var A: InfiniteArray<Waiter>
4     fun send(element: Message) { ... }
5     fun receive(): Message { ... }
6 }
7
8 struct Waiter<Message>(state: Any?, elem: Message?)
```

Листинг 2: предложенная архитектура рандеву-канала

S и R являются 64-битными счётчиками, содержащими количество соответственно отправок(S), получений(R). Предполагаем, что отправление сообщения производит рандеву с ожидающим получателем в случае $S < R$, блокирует в противном случае. Аналогично для получения сообщений. Массив A содержит заблокированные запросы. Каждый элемент представлен структурой `Waiter`: поле `state` содержит ожидающую сопрограмму, `elem` отправляемое сообщение.

Операции начинают исполнение с увеличения соответствующего им счётчика с помощью FAA, резервируя себе место в массиве. Далее они считывают счётчик противоположного типа, определяя свои дальнейшие действия: блокироваться или совершать рандеву. Если противоположный счётчик больше чем счётчик данной операции совершается рандеву; в противном случае в соответствующий элемент массива устанавливается сопрограмма и блокируется.

Синхронизация операций производится на поле `state`. Конечный автомат представлен на рисунке 1. Отправление сообщений располагает сообщением в поле `elem` перед синхронизацией, после этого `receive` может его безопасно забрать.

Изначально каждый элемент находится в состоянии `EMPTY` (можно выразить, например, используя `null` в коде). После этого, операция, которая будет блокироваться, меняет элемент на ожидающую сопрограмму. После этого операция противоположного вида возобновляет сохранённую сопрограмму, затем меняет состояние ячейки на `DONE`.

Однако, операция, которая должна возобновлять сохранённую сопрограмму, могла начать обрабатывать элемент когда он был в `EMPTY`, если сопрограмма ещё не успела быть сохранена. В случае такой гонки стратегии для `send` и `receive` разнятся. `send` изменяет состояние на `BUFFERED`; операция знает, что получатель уже на подходе, а значит нет причин ждать. Получатель же забирает элемент и не блокируется.

К сожалению, подобная стратегия не работает для `receive`, так как `receive` не только должен возобновлять сохранённую сопрограмму, но и получать сообщение. Вместо этого

операция “отравляет” ячейку, перемещая её в состояние **BROKEN**. Другие операции, находя ячейку в таком состоянии должны пропустить её и перезапустить работу. Стратегия перезапуска работы также применяется в случае, когда сопрограмма противоположной операции была отменена. В таком случае либо `tryUnpark` проваливается, либо ячейка уже находится в состоянии **INTERRUPTED**. В случае отмены операция должна изменить состояние ячейки на **INTERRUPTED**, чтобы избежать утечек памяти. Это можно сделать, к примеру, через механизм `onInterrupt`, являющийся параметром метода `park`.

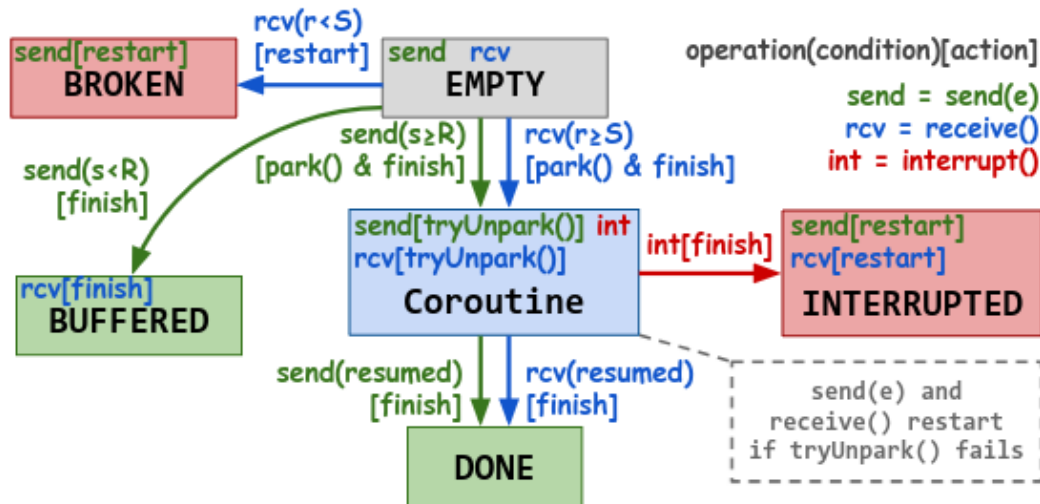


Рис. 1: Диаграмма конечного автомата, используемого операциями для синхронизации

3.1 Отправление сообщений

Чтобы отправить элемент, операция сначала инкрементирует счётчик `S` с помощью `FAA`, получая значение перед инкрементом. Затем, помещает сообщение в `A[S]`, и, наконец, обновляет ячейку согласно диаграмме, изображённой на рисунке 1, используя метод `updCellSend`.

Функция `updCellSend` написана в виде автомата специально, чтобы быть консистентной с диаграммой. Логика функции выполняется в бесконечном цикле, получая значение обрабатываемой ячейки и счётчика `R` в начале функции.

Когда ячейка находится в состоянии **EMPTY**, а также $S \geq R$ (означая, что операция получателя ещё не приходит), операция решает заблокироваться. Таким образом, операция получает ссылку на текущую сопрогammu, обменивая её с **EMPTY** с помощью атомарной инструкции `CAS`, затем блокирует сопрогammu через `cor.park`. Если `CAS` проваливается, операция перезапускается. Операция успешно завершается после возобновления получателем. В альтернативном исходе событий операция `send` могла быть отменена; в таком случае будет исполнен обработчик, переданный параметром метода `park`, который переведёт клетку в состояние **INTERRUPTED**, а также очистит поле сообщения.

Мы знаем, что когда клетка содержит сопрогammu, она ожидает `receive`, чтобы провести с ним randevu. В таком случае `send` пытается возобновить сопрогammu, вызывая `tryUnpark`. При успехе состояние клетки переводится в **DONE**, а операция успешно завершается. Иначе, если получатель прерван, операция счищает сообщение, чтобы из-

бежать утечки памяти, а после перезапускается. Такие же действия актуальны и когда операция находит клетку в состоянии INTERRUPTED или BROKEN.

```

1 fun send(element: E) = while (true) {
2   s := FAA(&S, +1) // reserve a cell
3   A[s].elem = element
4   if updCellSend(s): return
5 }
6 // Returns `false` if this 'send(e)' should restart
7 fun updCellSend(s: Int): Bool = while (true) {
8   state := A[s].state // read the current state
9   r := R // read the receiver 's counter
10  when {
11    // Empty and no receiver is coming => suspend
12    state == null && s >= r:
13      cor := curCor() // get the current coroutine
14      if CAS(&A[s].state, null, cor):
15        cor.park( // wait for a rendezvous
16          onInterrupt = {A[s] = (INTERRUPTED, null)})
17      return true
18    // Waiting receiver => try to resume it
19    state is Coroutine:
20      if state.tryUnpark():
21        A[s].state = DONE ; return true
22      else: // interrupted , clean the cell and fail
23        A[s].elem = null ; return false
24    // Empty but a receiver is coming => elimination
25    state == null && s < r:
26      if CAS (&A[s].state , null , BUFFERED ): return true
27    // Interrupted receiver or poisoned => fail
28    state == INTERRUPTED || state == BROKEN :
29      A[s].elem = null // clean to avoid memory leaks
30    return false
31  }
32 }

```

Листинг 3: алгоритм для операции send

3.2 Получение сообщений

Реализация операции `receive` является симметричной и обновляет состояние ячейки по диаграмме на рисунке 1. Отличиями является то, что `receive` помечает ячейку `BROKEN` в случае, когда она находится в состоянии `EMPTY`, но randеву должно произойти, а также проверяет ячейку на состояние `BUFFERED`.


```

1 fun receive(): E = while (true) {
2   r := FAA (&R , +1 ) // reserve a cell
3   if updCellRcv (r):
4     e := A[r].elem; A[r].elem = null; return e
5 }
6 // Returns `false` if this 'receive()' should restart
7 fun updCellRcv(r: Int): Bool = while (true) {
8   state := A[r].state // read the current state
9   s := S // read the sender 's' counter
10  when {
11    // Empty and no sender is coming => suspend
12    state == null && r >= s:
13      cor := curCor () // get the current coroutine
14      if CAS(&A[r].state, null, cor):
15        cor.park( // wait for a rendezvous
16          onInterrupt = {A[r].state = INTERRUPTED })
17      return true
18    // Waiting sender => try to resume it
19    state is Coroutine:
20      if state.tryUnpark():
21        A[r].state = DONE ; return true
22      else: // interrupted
23        return false
24    // Empty but a sender is coming => poison
25    state == null && r < s:
26      if CAS(&A[r].state, null, BROKEN): return false
27      // An elimination has happened => finish
28    state == BUFFERED: return true
29    // Interrupted sender => fail
30    state == INTERRUPTED: return false
31  }
32 }

```

Листинг 4: алгоритм для операции receive

4 Измерения

Данный алгоритм был реализован и интегрирован в стандартную библиотеку Kotlin Coroutines. Для сравнения производительности была использована стандартная реализация в библиотеке Kotlin Coroutines, а также алгоритмы Scherer[2], и Koval[3].

4.1 Окружение

Эксперименты запускались на сервере с 4 Intel Xeon Gold 5128 (Cascade Lake) с 16-ядерными сокетами. Всего при запуске использовалось 128 потоков.

4.2 Бенчмарки

Для сравнения производительности использовался классический паттерн производителя-потребителя: несколько сопрограмм используют один и тот же рандеву-канал, используя `send` и `receive` на нём. Используется одинаковое количество производителей и потребителей, оно либо равняется количеству потоков или является фиксированным числом до 1000. Замеряется время, необходимое для передачи 10^6 сообщений, а также пропускная способность. Также симулируется фоновая работа между операций: потребляется в среднем 100 итераций циклов(следуя геометрическому распределению), чтобы уменьшить конкуренцию за канал.

4.3 Результаты

Результаты измерений изображены на рисунке 2. Стоит заметить масштабируемость: в то время как пропускная способность остальных алгоритмов деградирует с увеличением количества потоков, предложенное решение продолжает масштабироваться.

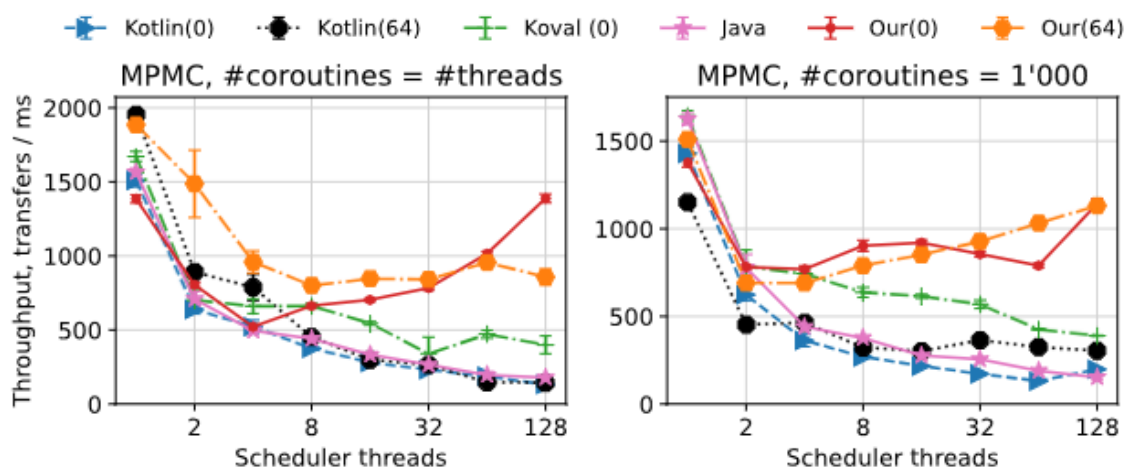


Рис. 2: результаты измерений

4.4 Отравленные ячейки

Также была собрана статистика, касающаяся количества отравленных(BROKEN) ячеек. Наблюдения показали, что оно никогда не превышает 10% от количества ячеек.

4.5 Использование памяти

Дополнительно были собраны данные о давлении на выделение памяти. При низкой конкуренции рандеву-канал имеет такие же показатели как алгоритм Koval et al., т.к. они оба используют связный список сегментов. Синхронизированная очередь Java и стандартные механизмы в Kotlin показывают 40% и 115% больше накладных расходов соответственно. При высокой конкуренции предложенный алгоритм является наилучшим, в то время как остальные решения имеют расходы выше от 20% (Java) до 90% (Kotlin).

5 Заключение

Предложенная реализация рандеву-каналов устраняет деградацию производительности и позволяет масштабировать передачу сообщений. У неё есть следующие достоинства:

- Алгоритм учитывает отмену сопрограмм.
- Алгоритм отлично масштабируется при конкуренции

Также можно заметить следующие слабости:

- Алгоритм явно рассчитывает на наличие сборки мусора в языке
- Отравленные состояния ячеек заставляют алгоритм перезапускаться, что делает задержку непредсказуемой.

Список литературы

- [1] Nikita Koval, Dan Alistarh, Roman Elizarov. *Fast and Scalable Channels in Kotlin Coroutines*.
- [2] William N Scherer III, Doug Lea, and Michael L Scott. *Scalable synchronous queues*.
- [3] Nikita Koval, Dan Alistarh, and Roman Elizarov. *Scalable fifo channels for programming via communicating sequential processes*.